03 de 20 - T2 03/03/2026 - T1 05/03/2026

RECAPTULANDO

- Solicite para o usuário digitar no terminar seu nome e sua idade.

- Mostre no terminal os dados lidos no seguinte formato:
  - **Nome lido**, você tem **idade lida** anos.

- Método **Console.ReadLine()**, disponível no namespace **System**.

- Caracteres interpolados.

```
internal class Program
{
    private static void Main(string[] args)
    {
        // Mostra uma mensagem solicitando o nome
        Console.Write("Digite seu nome: ");
        // Realiza a leitura e atribuição na variável nome
        string nome = Console.ReadLine();

        // Mostra uma mensagem solicitando a idade
        // Realiza a leitura e atribuição na variável idade
        Console.Write("Digite sua idade: ");
        string idade = Console.ReadLine();

        // Estamos mostrando para o usuário seu nome e idade.
        // Utilizamos o recurso cadeias de caracteres interpoladas $ {} {}
        Console.WriteLine($"{Environment.NewLine}{nome}, você tem {idade} anos.");
    }
}
```

Interpolação - **$"{}"**

https://learn.microsoft.com/pt-br/dotnet/csharp/tutorials/string-interpolation

- O recurso interpolação de cadeia de caracteres baseia-se no recurso formatação composta e fornece uma sintaxe mais legível e conveniente para incluir resultados de expressão formatada em uma cadeia de caracteres de resultado.
- Para identificar uma literal de cadeia de caracteres como uma cadeia de caracteres interpolada, preceda-o com o símbolo **$**.
- **Você pode inserir qualquer expressão C# válida que retorna um valor em uma cadeia de caracteres interpolada.**

```
Console.WriteLine($"{Environment.NewLine}{nome}, você tem {idade} anos.");
```

```
Console.WriteLine($"Resultado: {90 + 200 / 5}.");
```

Avisos de análise de código

- Error/Erros:
  - Tipos incompatíveis
  - Sintaxe errada
  - etc

- Warning/Avisos:
  - Variáveis declaradas e não utilizadas
  - Possíveis valores nulos
  - Recurso deprecated/descontinuado
  - etc

Parar! Analisar o código do erro/aviso gerado e realizar a solução!

Simples assim!

```
0 references
private static voi
{
    // Mostra uma
    Console.Write(
    // Realiza a l
    string nome = Console.ReadLine();
```

class System.Console

Represents the standard input, output, and error streams for console applications. This class cannot be inherited.

Conversão de literal nula ou possível valor nulo em tipo não anulável.
[NomeIdade] csharp(CS8600)

Exibir o Problema (Alt+F8) Nenhuma correção rápida disponível

PROBLEMAS 2 SAÍDA TERMINAL CONSOLE DE DEPURAÇÃO

Program.cs 2

- Conversão de literal nula ou possível valor nulo em tipo não anulável. [NomeIdade] csharp(CS8600) Ln 8, Col 23
- Conversão de literal nula ou possível valor nulo em tipo não anulável. [NomeIdade] csharp(CS8600) Ln 15, Col 24

0 2 NomeIdade

```
PS C:\Users\Luciano\OneDrive - UDESC Universidade do Estado de Santa Catarina\PESSOAL\WWW\c#\POO\NomeIdade> dotnet run
C:\Users\Luciano\OneDrive - UDESC Universidade do Estado de Santa Catarina\PESSOAL\WWW\c#\POO\NomeIdade\Program.cs(8,23): warning CS8600: Conversão de literal nula ou possível valor
 nulo em tipo não anulável. [C:\Users\Luciano\OneDrive - UDESC Universidade do Estado de Santa Catarina\PESSOAL\WWW\c#\POO\NomeIdade\NomeIdade.csproj]
C:\Users\Luciano\OneDrive - UDESC Universidade do Estado de Santa Catarina\PESSOAL\WWW\c#\POO\NomeIdade\Program.cs(15,24): warning CS8600: Conversão de literal nula ou possível valo
r nulo em tipo não anulável. [C:\Users\Luciano\OneDrive - UDESC Universidade do Estado de Santa Catarina\PESSOAL\WWW\c#\POO\NomeIdade\NomeIdade.csproj]
Digite seu nome: Luciano
Digite sua idade: 47

Luciano, você tem 47 anos.
```

Tipos de referência anuláveis

https://learn.microsoft.com/en-us/dotnet/csharp/nullable-references

- Em um contexto alheio anulável, todos os tipos de referência eram anuláveis.
- Tipos de referência anuláveis referem-se a um grupo de recursos ativados em um contexto com **reconhecimento de valor nulo** que minimiza a probabilidade de seu código fazer com que o tempo de execução lance System.NullReferenceException.
- Os tipos de referência anuláveis incluem três recursos que ajudam a evitar essas exceções, incluindo a capacidade de marcar explicitamente um tipo de referência como anulável:
  - Análise de fluxo estático aprimorada que determina se uma variável pode ser nullantes de desreferenciá-la.
  - Atributos que anotam APIs para que a análise de fluxo determine o estado nulo.
  - Anotações de variáveis que os desenvolvedores usam para declarar explicitamente o estado nulo pretendido para uma variável.
- A análise de estado nulo e as anotações de variáveis são desativadas por padrão para projetos existentes, o que significa que todos os tipos de referência continuam anuláveis.
- A partir do .NET 6, eles são habilitados por padrão para novos projetos.

- Em nossos códigos atuais não precisaremos ainda dos recurso **Nullable**, então podemos desativar ele de forma global em nosso projeto.
- Podemos realizar isso de duas formas:
  - Incluindo uma diretiva em **Program.cs**
  - Editando o arquivo de configuração do projeto, **Arquivo.csproj**

```
#nullable disable
```

```
<Project Sdk="Microsoft.NET.Sdk">


  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>disable</Nullable>
  </PropertyGroup>


</Project>
```

- Em projeto onde o recurso **Nullable** é utilizado, é possível realizar **anotações de variáveis anuláveis individualmente**, utilizando o caractere **?** para indicar que a variável aceita valores nulos, e o caractere **!** para indicar que a variável não é nula.

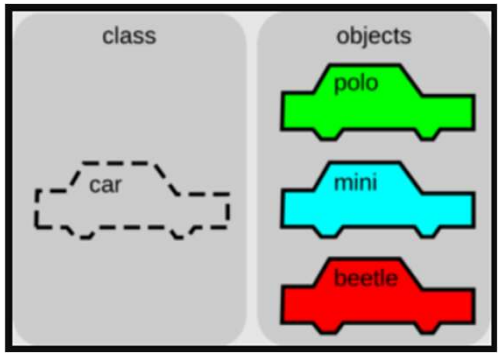
```
string? nome = Console.ReadLine();

string? idade = Console.ReadLine();

Console.WriteLine($"{nome} { int.Parse(idade!) }");
```

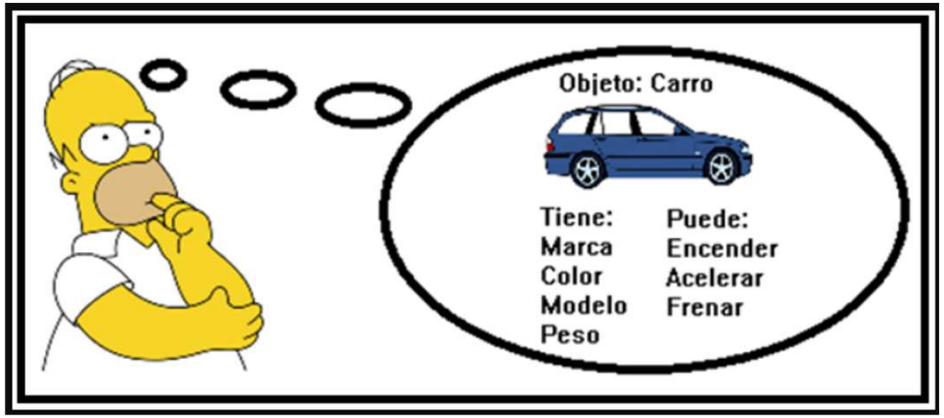
KEEP
CALM
QUE HOJE É
DIA DE
MALDADE

Que
comecem
os jogos.

KEEP
CALM
AND
PROGRAME
ORIENTADO A OBJETOS

POO
ENCAPSULAMENTO
ABSTRAÇÃO
HERANÇA
POLIMORFISMO


class
objects
car
polo
mini
beetle


Objeto: Carro
Tiene:
Marca
Color
Modelo
Peso
Puede:
Encender
Acelerar
Frenar

Namespaces

https://learn.microsoft.com/pt-br/dotnet/csharp/fundamentals/types/namespace

- Com o crescimento dos sistemas, passamos a ter diversas classes nele.
- Por exemplo, em nosso pequeno exemplo temos as classes:
  - Program, Livro e Leitor.
- O grande problema que surge com os sistemas grandes é a organização de todas as suas classes.
- Para evitar que o sistema fique caótico, podemos **agrupar as classes por características comuns** e dar um nome para cada um desses grupos.
- Isto é, agruparíamos um conjunto de classes em um espaço em comum e lhe daríamos um nome, como por exemplo **Biblioteca**.
- Esse espaço definido por um nome é chamado de **namespace**.

- Namespaces em C# são conjuntos de identificadores que tem como função agrupar funcionalidades comuns assim também organizando-as.

- Namespaces também prestam assistência em nome evitando confrontos entre os dois conjuntos de código.

- Implementar Namespaces no seu código é um bom hábito, pois pode lhe poupar de problemas futuros, quando você quiser reutilizar alguns de seus códigos.

- Segundo a convenção de nomes adotada pela Microsoft (http://msdn.microsoft.com/en-us/library/893ke618.aspx), os **namespaces** devem ter a forma:

  NomeDaEmpresa.NomeDoProjeto.ModuloDoSistema

- No nosso exemplo, os namespaces poderiam ficar da seguinte forma:

  Uniplac.POO.Biblioteca

- Mas nesse momento, precisamos somente entender que todas as nossas classes de uma solução, devem fazer parte de um mesmo namespace, ai no decorrer das aulas, conforme necessidade, vamos vendo outros casos de uso.

VAMOS FAZER

JUNTOS

Exemplo

- O paradigma orientado a objetos (POO) estrutura o código-fonte de um programa seguindo os **objetos** do **mundo real**.
- Se pegarmos como exemplo um **livro**, como podemos representar este objeto do mundo real para uma linguagem de programação, no nosso caso **C#**?

```
namespace Biblioteca
{

    public class Livro
    {


    }

}
```

- Em POO, cada **objeto** do **mundo real** é representado através de uma **classe**.
- Então, utilizando C#, podemos começar a representar esse livro criando a classe **Livro.cs**.

- Com isso o computador já **sabe** que existe um livro.
- Entretanto, só isso não basta.
- Precisamos fornecer mais informações sobre esse livro.

- Como podemos descrever as **características** que esse livro possui?

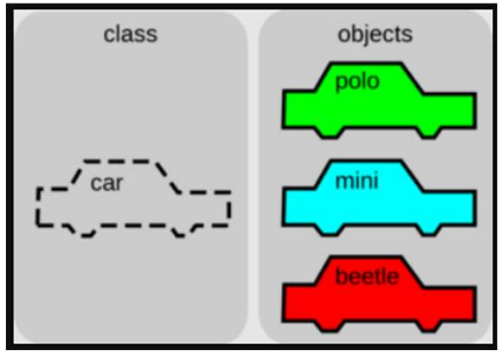
- Usamos os **atributos da classe** para definir o que um Livro possui.
  - Chamamos de **atributos** as **características** dos nossos objetos, que são **variáveis** que pertencem àquela classe e descrevem suas características.
- O objetivo aqui é definir **tudo que um Livro tem**, definindo assim seus **atributos**.
- Com isso já estamos representando de forma mais precisa um objeto livro do mundo real.
- Através da nossa classe Livro, conseguimos modelar um livro do mundo real de uma forma que o computador consiga interpretar.

```
namespace Biblioteca;

public class Livro
{
    public string Titulo;
    public string SubTitulo;
    public string Escritor;
    public string Editora;
    public string Genero;
    public int AnoPublicacao;
    public string TipoDaCapa;
    public int NumeroDePaginas;
}
```

- **A orientação a objetos, representa o mundo real em forma de objetos.**

Construindo e utilizando um objeto no sistema

- No passo anterior criamos uma **classe** que representa um Livro, ou seja, criamos um **modelo**, um **projeto**.
- Nossa classe informa para o computador **o que é um livro**, mas **não que existe um**, é apenas uma **descrição do objeto** e não o objeto em si.
- Assim, com base nessa **classe**, utilizando a mesma descrição, podemos criar vários objetos **Livro** na memória do computador, e cada **objeto** criado vai representar um **Livro** que realmente existe.



- Então para criamos um **objeto computacional**, precisamos pegar uma classe e **criar esse objeto na memória do computador**, utilizando para isso o conceito de **instância**.
  - **Instância** significa um objeto criado a partir de uma classe na memória do computador.

- Para utilizar uma classe devemos **declarar uma variável do seu tipo**.
- Na sequencia, para criar o objeto, precisamos realizar a sua **construção ou instanciação**, usamos a palavra chave **new** seguida do nome da Classe() para realizar a **instância**.
- No código ao lado, utilizando a mesma classe, estamos criando três instancias.
- Cada uma dessas instancias representa um **Livro** de verdade e é totalmente independente das outras, somente utilizaram o mesmo modelo para serem construídas.
- Vejam que para cada objeto criado temos uma **variável referenciando** ele (**l1**, **l2** e **l3**).
- Essas variáveis são chamadas de **variáveis de instância**, e guardam os objetos criados.
- É através da variável de instancia que conseguiremos acessar os dados do nosso objeto.

```
namespace Biblioteca;

internal class Program
{
    Program()
    {
        Livro l1;
        l1 = new Livro();

        Livro l2 = new Livro();

        Livro l3 = new Livro();
    }

    private static void Main(string[] args)
    {
        _ = new Program();
    }
}
```

```
namespace Biblioteca;

internal class Program
{
    Program()
    {
        Livro l1 = new Livro();
        l1.Titulo = "O contador de Histórias";
        l1.SubTitulo = "Memórias de vida e música";
        l1.Escritor = "Dave Grohl";

        Livro l2 = new Livro();
        l2.Titulo = "O guia do Mochileiro das Galáxias";
        l2.SubTitulo = "Não entre em pânico";
        l2.Escritor = "Douglas Adams";

        Livro l3 = new Livro();
        l3.Titulo = "Memórias de um legionário";
        l3.Escritor = "Dado Villa-Lobos";

        Console.WriteLine($"{l1.Titulo} - {l1.Escritor}");
        Console.WriteLine($"{l2.Titulo} - {l2.Escritor}");
        Console.WriteLine($"{l3.Titulo} - {l3.Escritor}");
    }

    private static void Main(string[] args)
    {
        _ = new Program();
    }
}
```

```
l3.
AnoPublicacao
Editora
Equals
Escritor
Genero
GetHashCode
GetType
NumeroDePaginas
SubTitulo
TipoDaCapa
Titulo
```

- Agora que já temos nossos objetos criados, podemos acessar/manipular seus atributos através das suas **variáveis de instancia**.

- Para isso basta usar: [variável de instancia] [ponto] [o que deseja acessar]

- É importante ressaltar que **cada instância possui seus atributos isolados das demais**.

- Isso significa que **l1** tem um atributo **Titulo**, e **l3** tem outro atributo **Titulo**, totalmente separados, independentes.

- Vamos melhorar nosso exemplo.
- Imagine o cenário de uma biblioteca pessoal de um leitor de livros, onde temos o **Leitor** e a sua **vasta coleção de Livros**.
- Como podemos modelar isso?

- Como já possuímos nossa classe Livro, temos que nos concentrar agora na criação de uma classe **Leitor**.

- **Quais são os atributos de um leitor?**

```
namespace Biblioteca;

public class Leitor
{
    public string Nome;
    public byte Idade;
    public List<Livro> LivrosLeitor = new List<Livro>();
}
```

- Assim como realizamos com o Livro, precisamos representar nosso Leitor através de uma classe, e nela definir seus atributos.
- Aqui precisaremos definir um atributo que nos permita **armazenar vários Livros do leitor.**
- Utilizaremos para isso a classe **List**, algo semelhante a um array.

ATENÇÃO
DESVIO

Material
complementar

Coleção Dinâmica

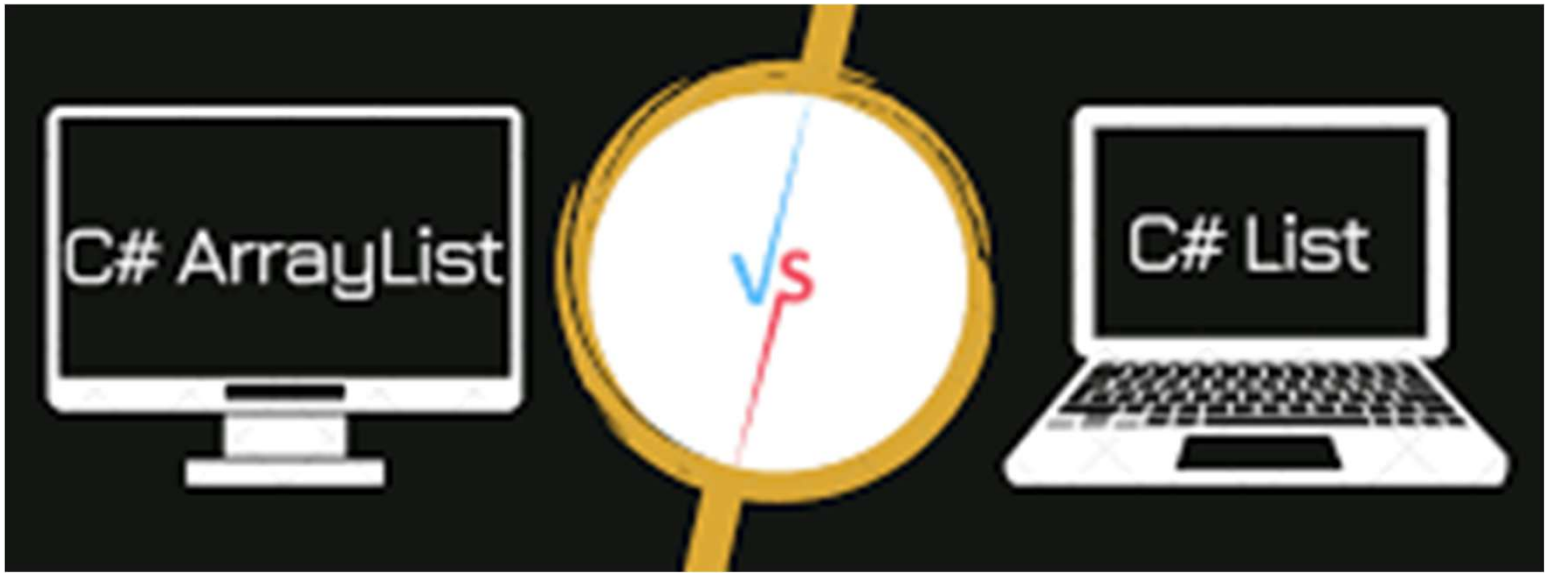
- Dada as características dos Arrays, há casos que precisamos usar **coleções** que permitam que o usuário possa inserir a quantidade de dados que ele desejar, sem que nos preocupemos com o tamanho do Array.

- Temos para estes casos as classes **ArrayList** e **List**, por exemplo, integrantes respectivamente dos namespaces **System.Collections** e **System.Collections.Generic**, que são coleções de dados que, entre outras coisas, podem aumentar e diminuir dinamicamente.

Estrutura de Dados
Array, ArrayList e List<T>

https://www.delftstack.com/pt/howto/csharp/arraylist-vs-list-in-csharp/
http://www.linhadecodigo.com.br/artigo/2767/array-arraylist-e-listt-o-que-devemos-saber.aspx

- A Microsoft, através da FCL – Framework Class Library permite que o desenvolvedor crie Arrays, que são coleções de dados de um mesmo tipo.
- Uma característica dos Arrays é que eles são estáticos, ou seja, o programador não consegue aumentar a quantidade de posições em tempo de execução.
- Um Array declarado, ocupa posições adjacentes na memória RAM, com o mesmo nome e tipo.
- Na hierarquia de classes da FCL, podemos notar que um array herda da classe Array, do namespace System (**System.Array**), logo para podermos trabalhar com eles, basta incluir a referencia para o namespace System (using System;), o que por padrão já é feito.


C# ArrayList
VS
C# List

C# - classe **ArrayList**

https://learn.microsoft.com/pt-br/dotnet/api/system.collections.arraylist?view=net-7.0

- Os **ArrayList**s são coleções de dados que podem aumentar e diminuir dinamicamente, além de **permitir a inserção de qualquer tipo de dado**, no mesmo ArrayList (inteiros, Strings, Decimais e etc).

- Um arrayList herda da classe ArrayList, do namespace **System.Collections;** logo para podermos trabalhar com eles, basta incluir a referencia para esse namespace.

- No exemplo seguinte, criamos a instância da classe ArrayList - meuAL - e armazenamos e listamos nela diferentes valores de diferentes tipos de dados.

```
using System.Collections;

ArrayList meuAL = new ArrayList();

meuAL.Add(1);
meuAL.Add("Sistemas");
meuAL.Insert(1, "Uniplac");

Console.WriteLine("meuAL");
Console.WriteLine("Count: {0}", meuAL.Count );
Console.WriteLine("Capacity: {0}", meuAL.Capacity );
Console.WriteLine("Valores: " );

foreach (Object itemAtual in meuAL)
{
   Console.WriteLine(itemAtual);
}
```

```
Count: 3
Capacity: 4
Valores:
1
Uniplac
Sistemas
```

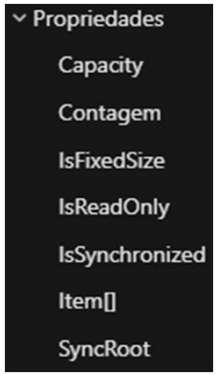
- Um **ArrayList** por permitir qualquer tipo de dado, sofre do problema de **Boxing** e **Unboxing**.

- Quando um item é **inserido** no ArrayList (**boxing**), ele é armazenado como tipo **object** automaticamente.

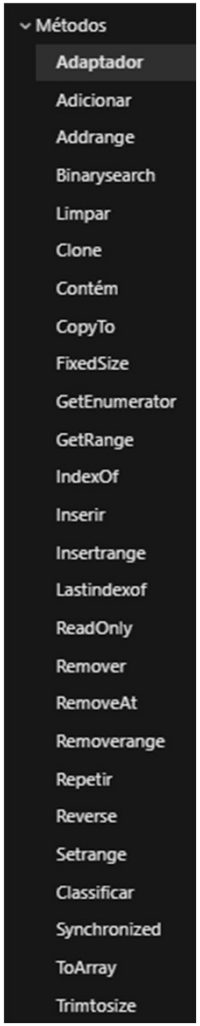
- Quando o usuário **retira** do ArrayList o dado (**unboxing**) ele é novamente convertido para o **tipo que era antes** de ir para o ArrayList.

- Essa conversões são internas ao ArrayList, o que diminui o seu desempenho em relação aos **List**(T), que não sofrem desse problema, já que em suas posições estão dados de um único tipo, dispensando essa análise.

- **Propriedades**
  - Capacity
    - Capacity é o número de elementos que o ArrayList pode armazenar.
    - Capacity é sempre maior ou igual a Count. Se Count exceder ao adicionar elementos Capacity, a capacidade será automaticamente aumentada realocando a matriz interna antes de copiar os elementos antigos e adicionar os novos elementos.
  - Count
    - Count é o número de elementos que estão realmente no ArrayList
  - IsFixedSize
    - true se o ArrayList tiver um valor fixo; caso contrário, false. O padrão é false.
  - IsReadOnly
    - true se o ArrayList for somente leitura; caso contrário, false. O padrão é false.
  - IsSynchronized
    - true caso o acesso ao ArrayList seja sincronizado (thread-safe); do contrário, false. O padrão é false.
  - Item[]
    - O índice com base em zero do elemento a ser obtido ou definido.
    - O elemento no índice especificado.
  - SyncRoot
    - Um objeto que pode ser usado para sincronizar o acesso à ArrayList.



- **Métodos**
  - Add
    - O objeto Object a ser adicionado ao **final** do ArrayList. O valor pode ser null.
    - Retorna o índice ArrayList ao qual o value foi adicionado.
  - Insert
    - Insere um elemento no ArrayList, no índice especificado.
  - Clear
    - Remove todos os elementos do ArrayList.
  - Clone
    - Cria uma cópia superficial do ArrayList.
    - Uma cópia superficial de uma coleção copia apenas os elementos da coleção, sejam eles tipos de referência ou tipos de valor, mas não copia os objetos aos quais as referências se referem. As referências na nova coleção apontam para os mesmos objetos aos quais as referências na coleção original apontam.
    - Por outro lado, uma cópia profunda de uma coleção copia os elementos e tudo referenciado direta ou indiretamente pelos elementos.
    - Esse método é uma O(n) operação, onde n está Count.
  - CopyTo
    - Copia o ArrayList, ou parte dele, para uma matriz unidimensional.
    - O Array unidimensional que é o destino dos elementos copiados de ArrayList. O Array deve ter indexação com base em zero.
  - Remove
    - Remove a primeira ocorrência de um objeto específico do ArrayList.
  - RemoveAt
    - Remove o elemento no índice especificado do ArrayList.
  - Reverse
    - Inverte a ordem dos elementos no ArrayList em uma parte dele.
  - Sort
    - Classifica os elementos no ArrayList ou parte dele.
  - ToArray
    - Copia os elementos do ArrayList em uma nova matriz.
  - TrimToSize
    - Define a capacidade para o número real de elementos em ArrayList.
  - IndexOf
    - Retorna o índice baseado em zero da primeira ocorrência de um valor no ArrayList ou em uma parte dele
  - LastIndexOf
    - Retorna o índice baseado em zero da última ocorrência de um valor no ArrayList ou em uma parte dele.



C# - classe **List**

https://learn.microsoft.com/pt-br/dotnet/api/system.collections.generic.list-1?view=net-7.0

- A classe **List(T)** é uma coleção que possui a mesma característica dos ArrayLists, que é a de ser **dinâmica**, e a característica dos Arrays que é de permitir um único tipo de dado.
- Como permite um único tipo de dado, não sofre o do problema de **Boxing** e **Unboxing** presente no **ArrayList.**
- As **List(T)** são coleções que herdam da classe List(T), do namespace: **System.Collections.Generics**, portanto para trabalhar com List(T), devemos incluir a referência ao namespace Generics.
- No exemplo seguinte, criamos a instância da classe List - meuL - que contém o tipo de dados inteiro e os valores inteiros armazenados.

```
using System.Collections;

List<int> meuL = new List<int>();

meuL.Add(1);
meuL.Add(2);
meuL.Insert(1, 0);

Console.WriteLine("meuL");
Console.WriteLine("Count: {0}", meuL.Count );
Console.WriteLine("Capacity: {0}", meuL.Capacity );
Console.WriteLine("Valores: " );

foreach (Object itemAtual in meuL)
{
   Console.WriteLine(itemAtual);
}
```

```
meuL
Count: 3
Capacity: 4
Valores:
1
0
2
```

- **Propriedades**
  - Capacity
    - Capacity é o número de elementos que o ArrayList pode armazenar.
    - Capacity é sempre maior ou igual a Count. Se Count exceder ao adicionar elementos Capacity, a capacidade será automaticamente aumentada realocando a matriz interna antes de copiar os elementos antigos e adicionar os novos elementos.
  - Count
    - Count é o número de elementos que estão realmente no ArrayList
  - Item[]
    - O índice com base em zero do elemento a ser obtido ou definido.
    - O elemento no índice especificado.

- **Métodos**
  - Add
    - Adiciona um objeto ao final do List<T>.
  - Insert
    - Insere um elemento no List<T>, no índice especificado.
  - Clear
    - Remove todos os elementos do List<T>.
  - CopyTo
    - Copia o List<T> ou parte dele para uma matriz.
  - Remove
    - Remove a primeira ocorrência de um objeto específico do List<T>.
  - RemoveAt
    - Remove o elemento no índice especificado do List<T>.
  - Reverse
    - Inverte a ordem dos elementos no List<T> em uma parte dele.
  - Sort
    - Classifica os elementos ou uma parte dos elementos na List<T> usando a implementação IComparer<T> especificada ou padrão, ou um delegado Comparison<T> fornecido para comparar os elementos de lista.
  - ToArray
    - Copia os elementos do List<T> em uma nova matriz.
  - TrimToSize
    - Define a capacidade para o número real de elementos em ArrayList.
  - IndexOf
    - Retorna o índice baseado em zero da primeira ocorrência de um valor no List<T> ou em uma parte dele.
  - LastIndexOf
    - Retorna o índice baseado em zero da última ocorrência de um valor no List<T> ou em uma parte dele.

Métodos
- Adicionar
- Addrange
- Asreadonly
- Binarysearch
- Limpar
- Contém
- Convertall
- CopyTo
- Ensurecapacity
- Exists
- Localizar
- Findall
- Findindex
- Findlast
- Findlastindex
- ForEach
- GetEnumerator
- GetRange
- IndexOf
- Inserir
- Insertrange
- Lastindexof
- Remover
- RemoveAll
- RemoveAt
- Removerange
- Reverse
- Classificar
- ToArray
- Trimexcess
- Trueforall

- No link abaixo, estou deixando a documentação de outras coleção da Microsoft, recomendo fortemente que olhem...

https://learn.microsoft.com/pt-br/dotnet/standard/collections/commonly-used-collection-types

RETORNO

```
namespace Biblioteca;

public class Leitor
{
    public string Nome;
    public byte Idade;
    public List<Livro> LivrosLeitor = new List<Livro>();

    public void AdicionarLivro(Livro livro)
    {
        this.LivrosLeitor.Add(livro);
    }

}
```

Dentro do método **AdicionarLivro()**, somente para exemplificar, utilizamos a palavra **this**.
Essa palavra referencia a instância em que esse método foi chamado.
Depois quando você chamar **leitor01.AdicionarLivro();** então this será o **leitor01**.

- Vimos até aqui que **objetos** são partes do mundo real, que **classes** representam esses objetos e que os **atributos** das classes representam **características** desses objetos.
- Porem, objetos além de possuírem características, também podem possuir **comportamentos**, ou seja, podem **fazer algo**, executar ações.

- Em nossa classe Leitor, por exemplo, queremos incluir um **comportamento** que permita ao **leitor adicionar na sua biblioteca um livro que ele possua**.
- Os comportamentos em POO são representados através dos **métodos**.

Tudo junto

- Conforme vimos anteriormente, as classes também podem ter **ações**, **comportamentos**, ou seja, **métodos** que representam **o que um objeto pode fazer**.
- Em nosso exemplo, agora já podemos **criar um leitor**, **alimentar seus atributos** e através do seu comportamento, ou seja, seu método, **adicionar em sua biblioteca os livros que ele possui**.
- Assim como ocorre com os atributos, todas as instancias de um objeto possuem os métodos que estejam disponíveis na classe.

```
namespace Biblioteca;

internal class Program
{
    Program()
    {
        Livro l1 = new Livro();
        l1.Titulo = "O contador de Histórias";
        l1.SubTitulo = "Memórias de vida e música";
        l1.Escritor = "Dave Grohl";

        Livro l2 = new Livro();
        l2.Titulo = "O guia do Mochileiro das Galáxias";
        l2.SubTitulo = "Não entre em pânico";
        l2.Escritor = "Douglas Adams";

        Livro l3 = new Livro();
        l3.Titulo = "Memórias de um legionário";
        l3.Escritor = "Dado Villa-Lobos";


        Leitor leitor1 = new Leitor();
        leitor1.Nome = "Luciano Coelho";
        leitor1.Idade = 17;

        leitor1.AdicionarLivro(l1);
        leitor1.AdicionarLivro(l2);
        leitor1.AdicionarLivro(l3);


        Leitor leitor2 = new Leitor();
        leitor2.Nome = "Abc Bolinhas";
        leitor2.Idade = 12;

        leitor2.AdicionarLivro(l3);

    }

    private static void Main(string[] args)
    {
        _ = new Program();
    }
}
```

- Veja como fica fácil se quisermos mostrar os dados do leitor:

```
Console.WriteLine($"{leitor1.Nome}, {leitor1.Idade} anos.");
Console.WriteLine($"Livros: ");
foreach (var livro in leitor1.LivrosLeitor)
{
   Console.WriteLine($"{livro.Titulo}, {livro.Escritor}");
}
```

- Claro que isso deve ser criado em um método, permitindo o uso em diferentes momentos da aplicação

ATENÇÃO
DESVIO

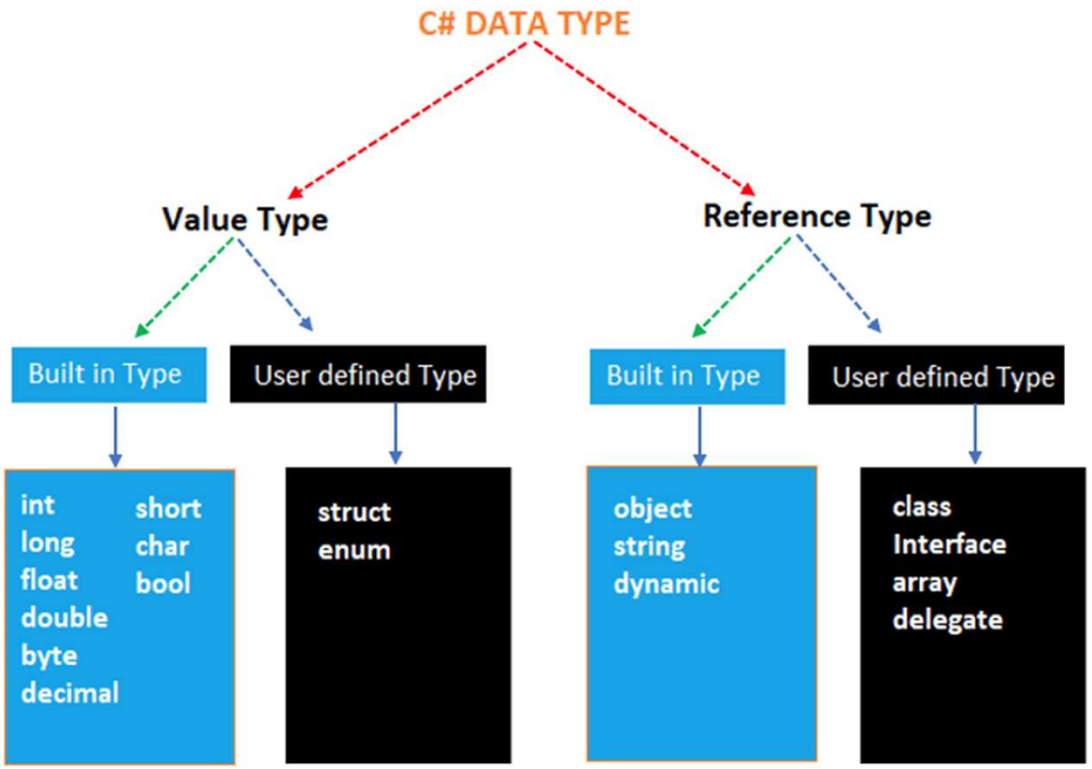
Material
complementar

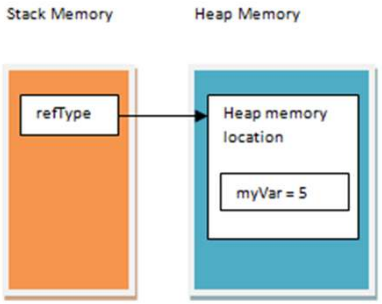
Reference Type

- Classes são **tipos de referencia**.

- Ou seja, na área **Stack** da memória da aplicação guardam apenas a **referência desse objeto**, ficando na área **Heap** seus valores.

- Com tipos de referência, duas ou mais variáveis podem fazer referência ao mesmo objeto.





Value

```
int teste = 2015;
int novoTeste = teste;
novoTeste = 1975;

Console.WriteLine($"{teste} , {novoTeste}");
```

2015 , 1975

Reference

```
Livro l3 = new Livro();
l3.Titulo = "Memórias de um legionário";
l3.Escritor = "Dado Villa-Lobos";
l3.AnoPublicacao = 2015;

Livro l4 = l3;
l4.AnoPublicacao = 1975;

Console.WriteLine($"{l3.AnoPublicacao} , {l4.AnoPublicacao}");
```

1975 , 1975

RETORNO

- E se um leitor quiser **doar** um livro para outro leitor?
- Usando o que conversamos sobre **referência** isso fica muito fácil!
- Basta que seja criado um novo comportamento/método na nossa classe Leitor que nos permita mover a referencia de um determinado livro do leitor A para o leitor B.

```
namespace Biblioteca;

public class Leitor
{
    public string Nome;
    public byte Idade;
    public List<Livro> LivrosLeitor = new List<Livro>();

    public void AdicionarLivro(Livro livro)
    {
        this.LivrosLeitor.Add(livro);
    }

    public void RemoverLivro(Livro livro)
    {
        this.LivrosLeitor.Remove(livro);
    }

    public void DoarLivro(Livro livroDoado, Leitor leitorDestino){
        leitorDestino.LivrosLeitor.Add(livroDoado);
        this.LivrosLeitor.Remove(livroDoado);
    }

}
```

```
namespace Biblioteca2;

internal class Program
{
    Program()
    {
        Livro l1 = new Livro();
        l1.Titulo = "O contador de Histórias";
        l1.SubTitulo = "Memórias de vida e música";
        l1.Escritor = "Dave Grohl";

        Livro l2 = new Livro();
        l2.Titulo = "O guia do Mochileiro das Galáxias";
        l2.SubTitulo = "Não entre em pânico";
        l2.Escritor = "Douglas Adams";

        Livro l3 = new Livro();
        l3.Titulo = "Memórias de um legionário";
        l3.Escritor = "Dado Villa-Lobos";

        Leitor leitor1 = new Leitor();
        leitor1.Nome = "Luciano Coelho";
        leitor1.Idade = 17;
        leitor1.AdicionarLivro(l1);
        leitor1.AdicionarLivro(l2);
        leitor1.AdicionarLivro(l3);

        Leitor leitor2 = new Leitor();
        leitor2.Nome = "Abc Bolinhas";
        leitor2.Idade = 12;
        leitor2.AdicionarLivro(l3);

        ListarLeitor(leitor1);

        leitor1.DoarLivro(l1, leitor2);

        ListarLeitor(leitor1);
        ListarLeitor(leitor2);
    }

    public void ListarLeitor(Leitor leitor)
    {
        Console.WriteLine($"\n{leitor.Nome}, {leitor.Idade} anos.");
        Console.WriteLine($"Livros: ");
        foreach (var livro in leitor.LivrosLeitor)
        {
            Console.WriteLine($"{livro.Titulo}, {livro.Escritor}");
        }
    }

    private static void Main(string[] args) { _ = new Program(); }
}
```